# Worksheet 10 — API Security (3 hrs)

> **Course:** Software Security (KOSEN69) · Week 10 **Aligned:** OWASP API Security Top 10:2023 — API1 BOLA · API3 Broken Object Property Level Auth (Mass Assignment) · API4 Unrestricted Resource Consumption **Signature game:** 🦹 crAPI Raid
>
> **Ethics note:** Run exploits **only** against the lab targets shipped here (`vulnerable_api.py` on `:8080`) or your own OWASP crAPI instance. Never test BOLA, mass assignment, or brute-force against systems you do not own or are not explicitly authorized to test.

## Part 1 — Student Information

| Name | Student ID | Date | Group |
|---|---|---|---|
| Siravit Thakaew | 6631503041 | 10/10/2569 | |

## Part 2 — Lecture Questions

1. Define **BOLA (API1:2023)**. Why is it the #1 API risk, and why can't a WAF reliably stop it?

BOLA (Broken Object Level Authorization) is a vulnerability that occurs when an API fails to adequately verify whether a user has the authorization to access a requested object. For example, a logged-in user named Alice might be able to change a URL from: /api/users/1/orders to /api/users/3/orders ...and then read Carol's order.

BOLA is the number one risk because APIs often expose various object IDs—such as `user_id`, `order_id`, or `document_id`—and many endpoints require reading or modifying this data. If object-level authorization checks are missing, an attacker could potentially access a large volume of data belonging to other users.

A WAF cannot reliably stop BOLA, as malicious requests often resemble legitimate ones—for example:

GET /api/users/3/orders HTTP/1.1

The request consists of valid HTTP and JSON, containing no SQL injection or payload patterns that a WAF would detect. The determination of whether the caller has permission to access user ID 3 must be handled within the application by verifying both the identity of the caller and their ownership of the object in every instance.

2. What is **mass assignment / broken object property level authorization (API3:2023)**? Contrast a blocklist vs. an allow-list of bindable fields.

Mass assignment means the server takes all fields from the request body and places them into an object or database, without restricting which fields the user can control.

For example, the client sends:

{ "username": "mallory", "password": "x", "is_admin": true, "balance": 1000000 }

If a server binds all fields directly, users can manually set values for fields like `is_admin` or `balance`, potentially leading to unauthorized privilege escalation or the modification of sensitive data.

A "blocklist" approach involves accepting all fields by default while explicitly excluding specific, sensitive ones—for example:

blocked = {"is_admin", "balance"}

This method carries risks; if new sensitive fields—such as `role`, `credit_limit`, or `verified`—are added in the future and one forgets to include them in the blocklist, those fields could become vulnerable to exploitation.

An "allow-list" approach, by contrast, explicitly defines which fields a client is permitted to submit—for example:

ALLOWED_CREATE_FIELDS = {"username", "password"}

In this case, the server only accepts fields included in the list, while sensitive fields like `id`, `balance`, and `is_admin` are managed exclusively by the server. The allow-list approach is more secure, as any newly added fields are not automatically exposed to client-side control.

3. **API4:2023** is unrestricted resource consumption. Give two distinct impacts (security and availability) of an unthrottled `/api/login`.

API4 refers to a vulnerability where an API allows users or attackers to consume resources without restriction—for example, by making repeated login requests without any limit on the number of attempts.

Key impacts include:

1.Brute-force or credential stuffing: Attackers can test a large number of passwords against a specific account (e.g., Alice's). If they discover the correct password, they can take over the account. The absence of rate limiting makes such attacks both rapid and low-cost.

2.Impact on availability (service disruption or slowdown): Sending a high volume of requests forces the server to consume excessive CPU, memory, database connections, and network bandwidth. This can degrade service for legitimate users or even lead to a Denial of Service (DoS).

Furthermore, this can result in higher costs, particularly if the API performs resource-intensive operations or utilizes services that charge on a per-request basis.

4. In `vulnerable_api.py`, `current_user()` trusts the `X-User-Id` header. Why is a client-supplied header NOT authentication, and what should replace it?

The `X-User-Id` header sent by a client does not constitute authentication, as clients can define or modify headers at will—for example, changing this:

X-User-Id: 1

to this:

X-User-Id: 3

If a server trusts such a value without verifying that the sender truly owns the identity, an attacker could easily impersonate another user or an administrator. Consequently, this header is merely a claim made by the client, not valid proof of identity.

In a production environment, you should employ authentication mechanisms that the server can independently verify, such as:

Session cookies that are randomly generated and tracked on the server side.

OAuth 2.0 / OpenID Connect.

Access tokens or JWTs that carry a valid signature and include verified claims (such as `iss`, `aud`, and `exp`), along with relevant permissions.

mTLS (mutual TLS) for service-to-service communication, where applicable.

Furthermore, even after authentication has confirmed that the user is, for instance, "Alice," you must still perform object-level authorization checks. For example, Alice might have permission to access `/api/users/1/orders` but lack the authorization to access `/api/users/3/orders`.

5. Why does object-level authorization have to be checked **per request, per object** rather than once at login?

Logging in merely confirms the identity of the caller; it does not imply that the caller has permission to access every object in the system.

A single user may have varying levels of access depending on factors such as:

The specific object being requested The owner of the data The user's assigned role The tenant or organization The status of the resource Changes in permissions during a session

For example, Alice might have permission to view her own orders, but not those belonging to Bob or Carol. Therefore, every request must verify the relationship between the caller and the requested object.

if caller["id"] != uid and not caller["is_admin"]: return jsonify({"error": "forbidden"}), 403

Checking identity only at the time of login confirms that "this is Alice," but it does not answer whether "Alice has permission to access this object." Consequently, authorization must be re-verified every time an individual object is accessed, modified, or deleted.

## Part 3 — Hands-on Lab (145 min)

**Learning goals:** Exploit BOLA, mass assignment, and a missing rate limit against a live API; then read the secure version and explain each fix.

**Prerequisites:** Docker + Docker Compose, `curl`. (Optional bonus: cloned OWASP crAPI.)

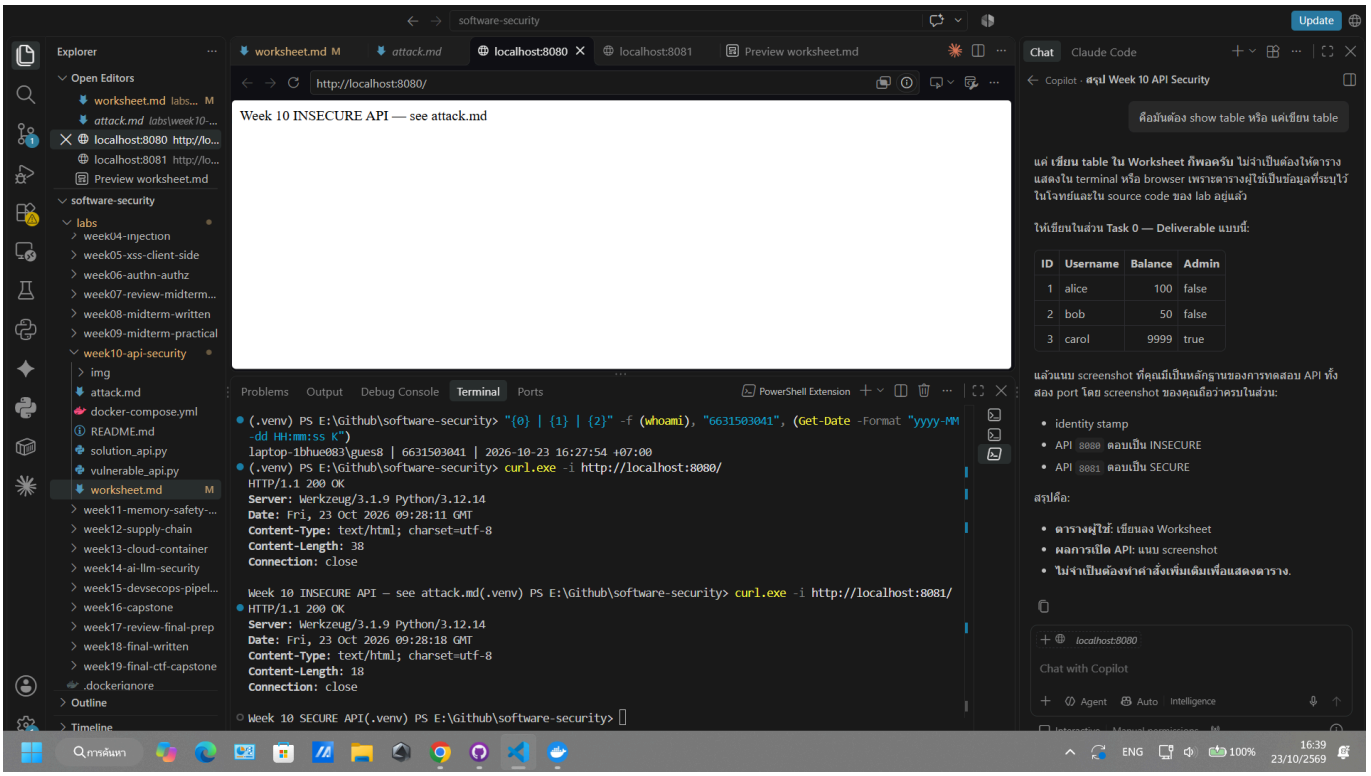
**Environment setup**

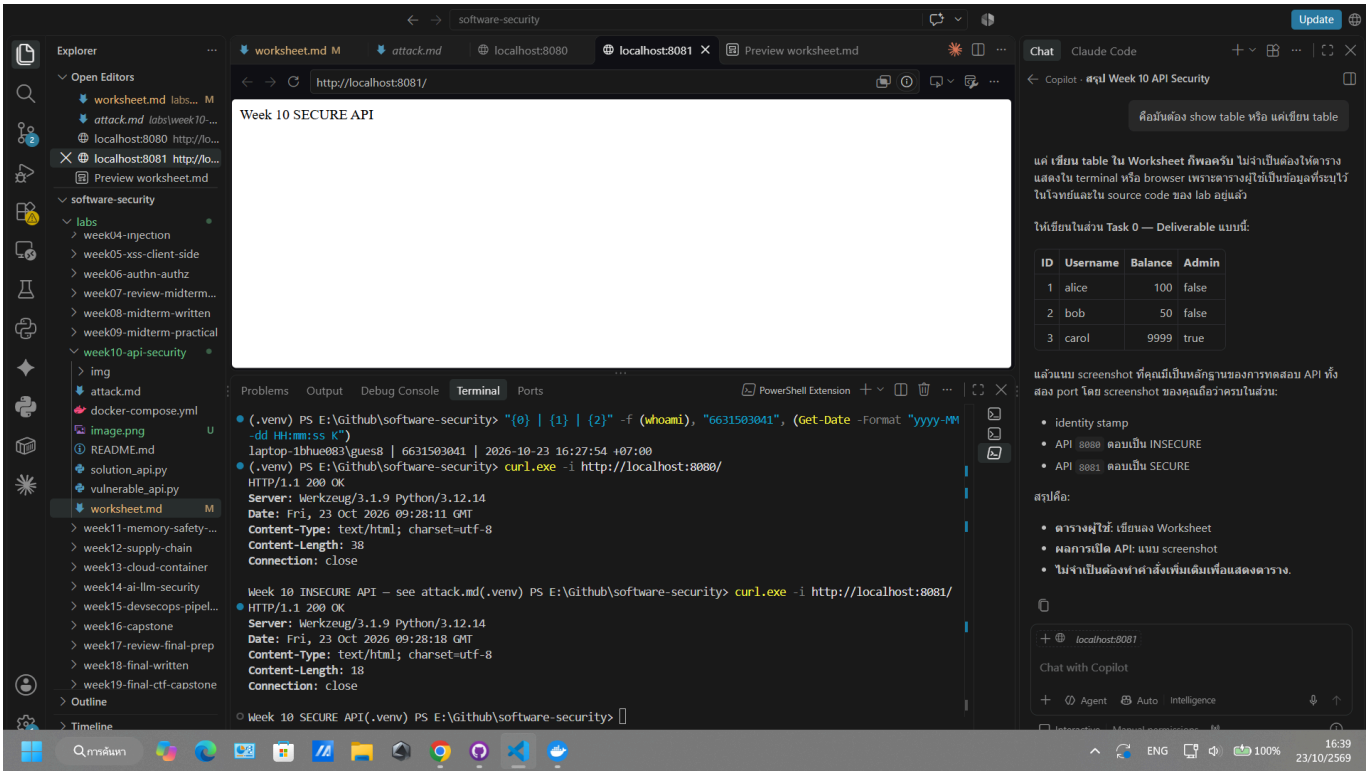
```
cd labs/week10-api-security
docker compose up         # INSECURE API on :8080, SECURE API on :8081
# Seeded users: alice(id 1) bob(id 2) carol(id 3, admin, balance 9999)
```

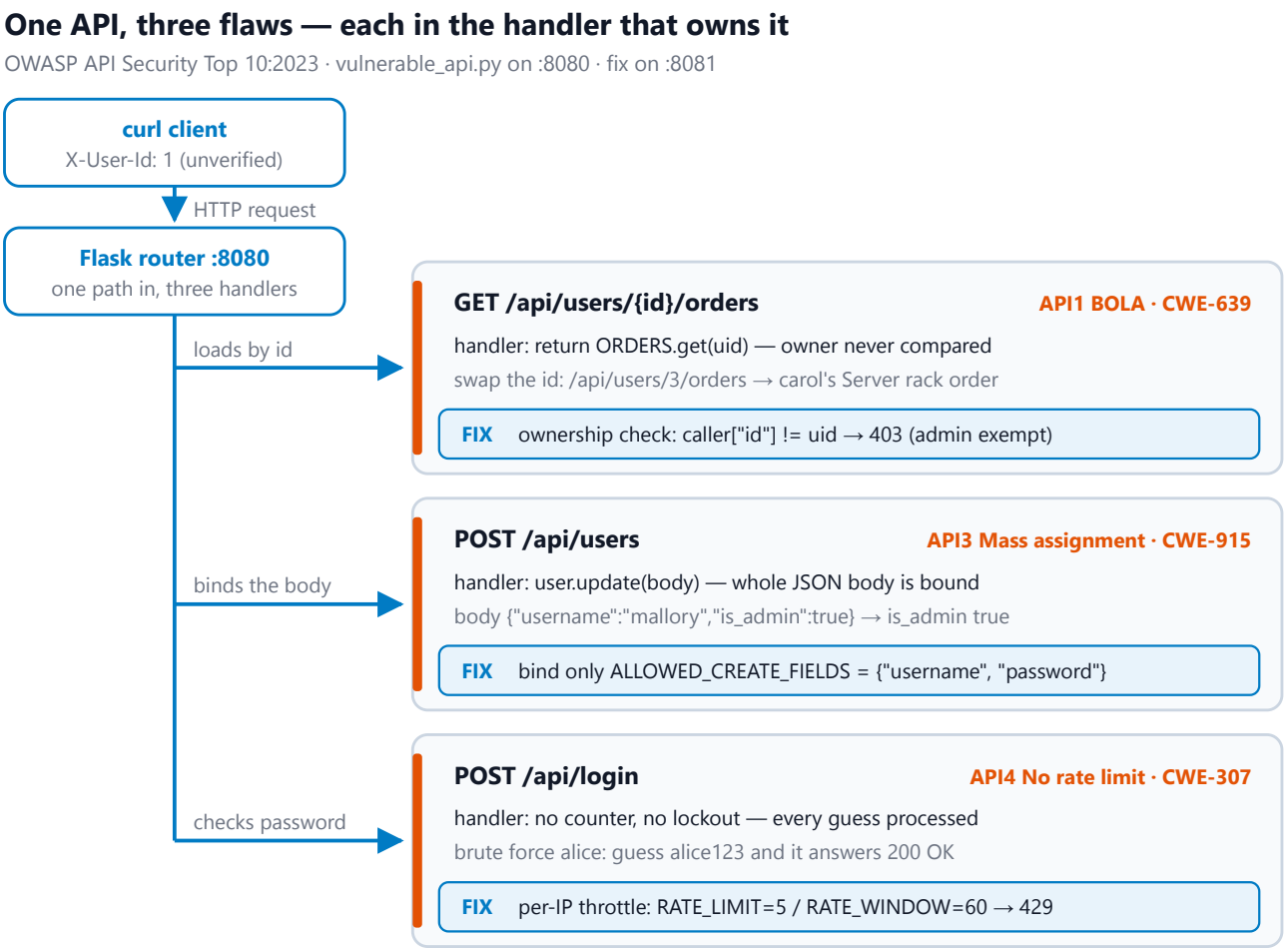
**What to submit per task:** the exact command(s) + raw response/HTTP status, a screenshot, and a 2–3 sentence mitigation note mapping the bug to its OWASP API id.

## Task 0 — Onboarding (15 min)

Confirm both APIs respond: `curl http://localhost:8080/` and `curl http://localhost:8081/`. Note which port is insecure. Record the three seeded users. **Deliverable:** both root responses + the user table.



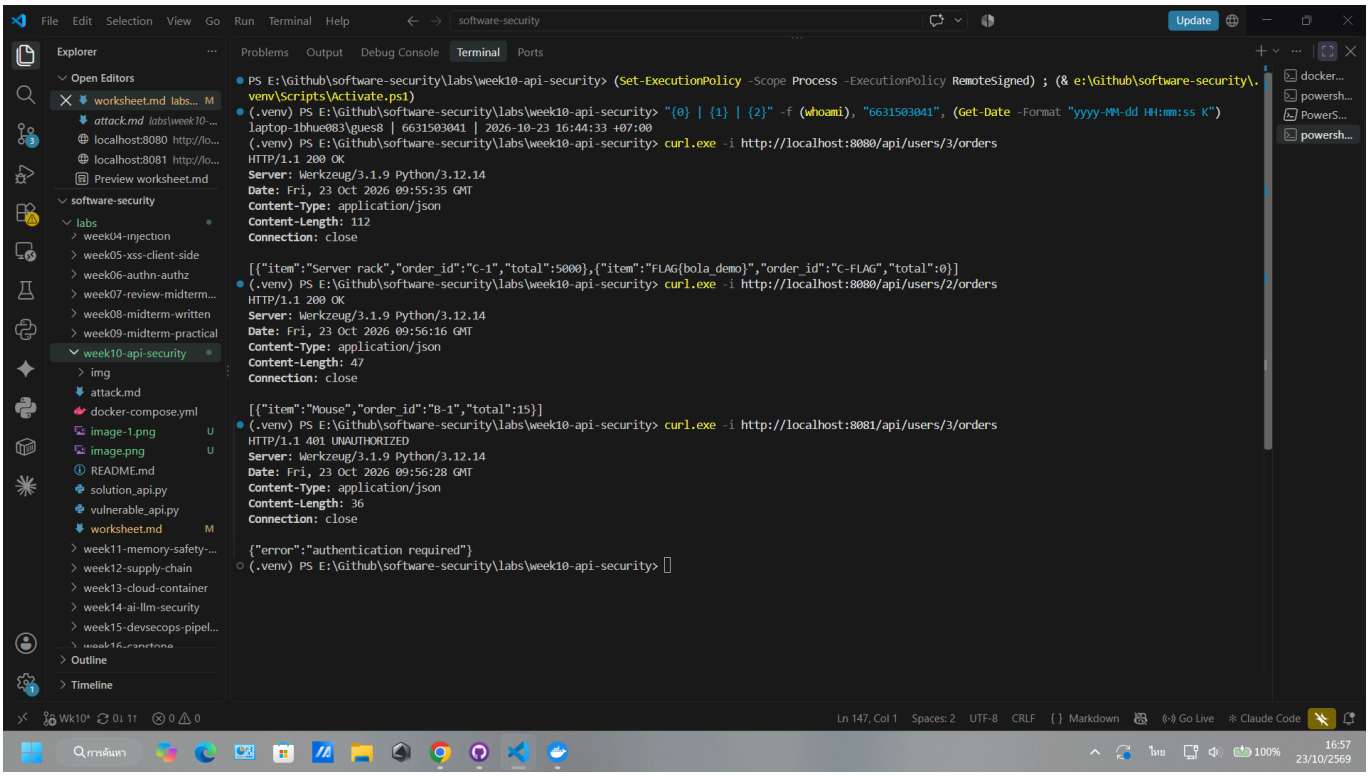


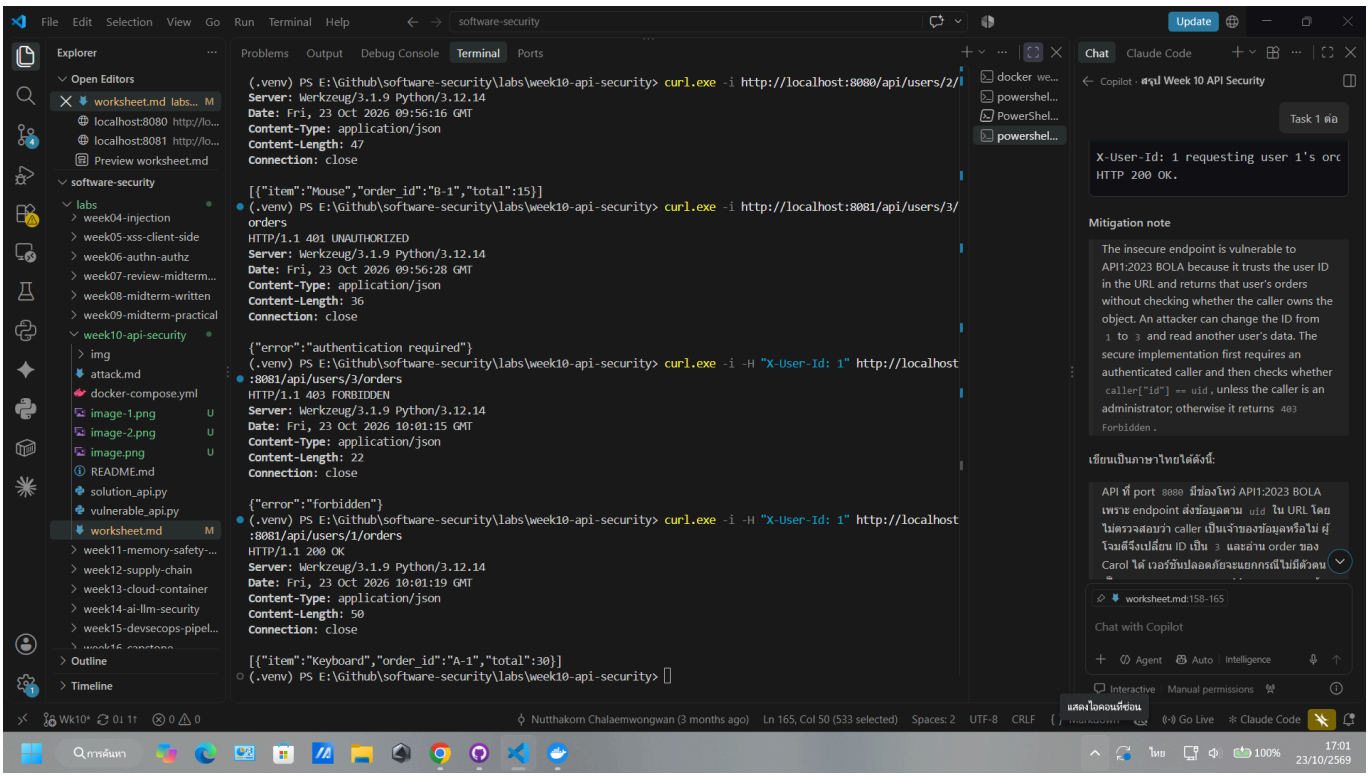

**All three are authorization and design flaws, not input flaws.**
The JSON is well-formed: escaping and encoding change nothing.
A WAF cannot know which order id belongs to which caller.

## Task 1 — BOLA: read another user's orders (35 min)

**Goal:** Read carol's (admin) orders without being carol — API1:2023. **Steps:**

1. `curl http://localhost:8080/api/users/3/orders` — note you receive the "Server rack" order with no auth.
2. Iterate the id: `curl http://localhost:8080/api/users/2/orders`.
3. On the secure API observe the ladder: `curl -i http://localhost:8081/api/users/3/orders` (401), `curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/3/orders` (403), `curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/1/orders` (200). **Deliverable:** the leaked order JSON + the 401/403/200 transcript + mitigation note.



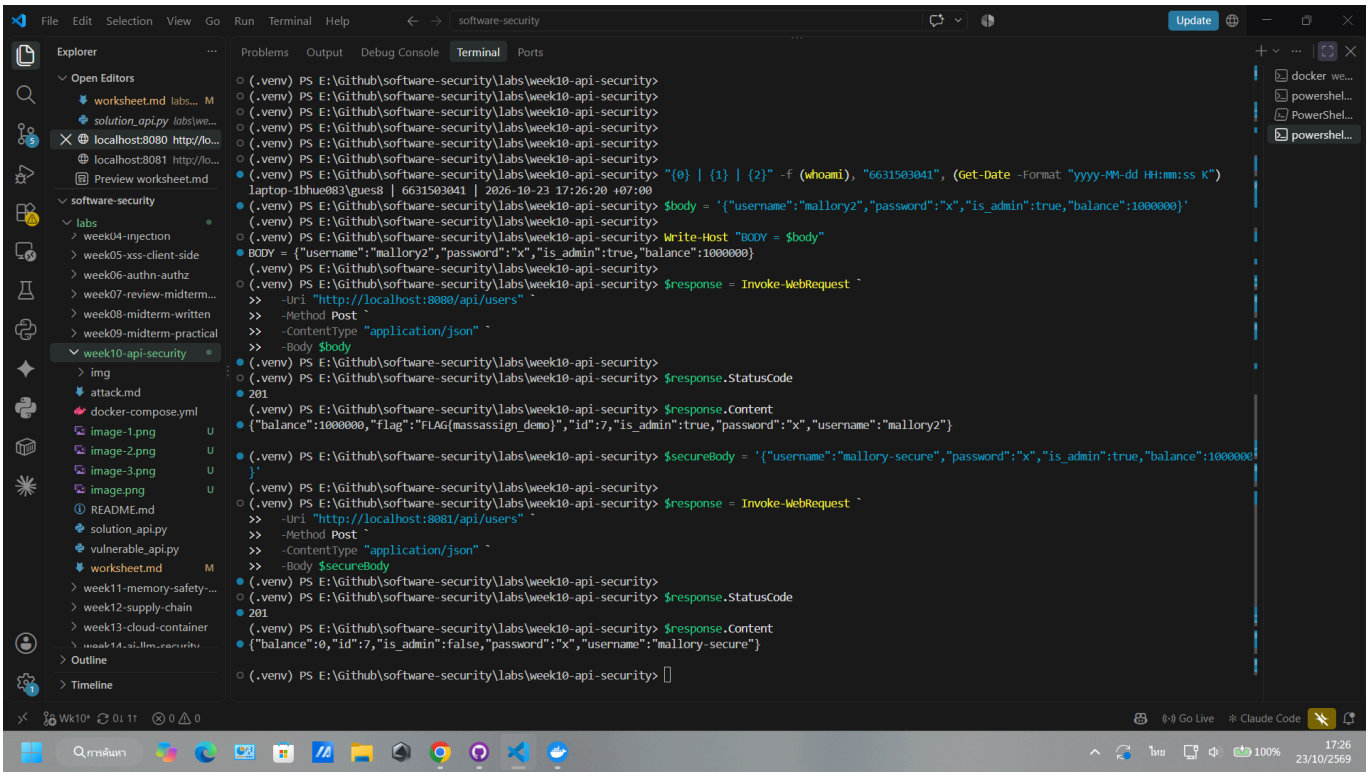

The insecure endpoint is vulnerable to API1:2023 BOLA because it trusts the user ID in the URL and returns that user's orders without checking whether the caller owns the object. An attacker can change the ID from 1 to 3 and read another user's data. The secure implementation first requires an authenticated caller and then checks whether caller["id"] == uid, unless the caller is an administrator; otherwise it returns 403 Forbidden.

## Task 2 — Mass assignment: self-promote to admin (30 min)

**Goal:** Smuggle `is_admin` + `balance` into user creation — API3:2023. **Steps:**

1. `curl -X POST http://localhost:8080/api/users -H "Content-Type: application/json" -d '{"username":"mallory","password":"x","is_admin":true,"balance":1000000}'`

2. Confirm the response echoes `"is_admin": true, "balance": 1000000`.
3. Repeat against `:8081` and confirm the smuggled fields are forced to `is_admin:false, balance:0`.
   **Deliverable:** both responses side by side + mitigation note (allow-list in `solution_api.py`).

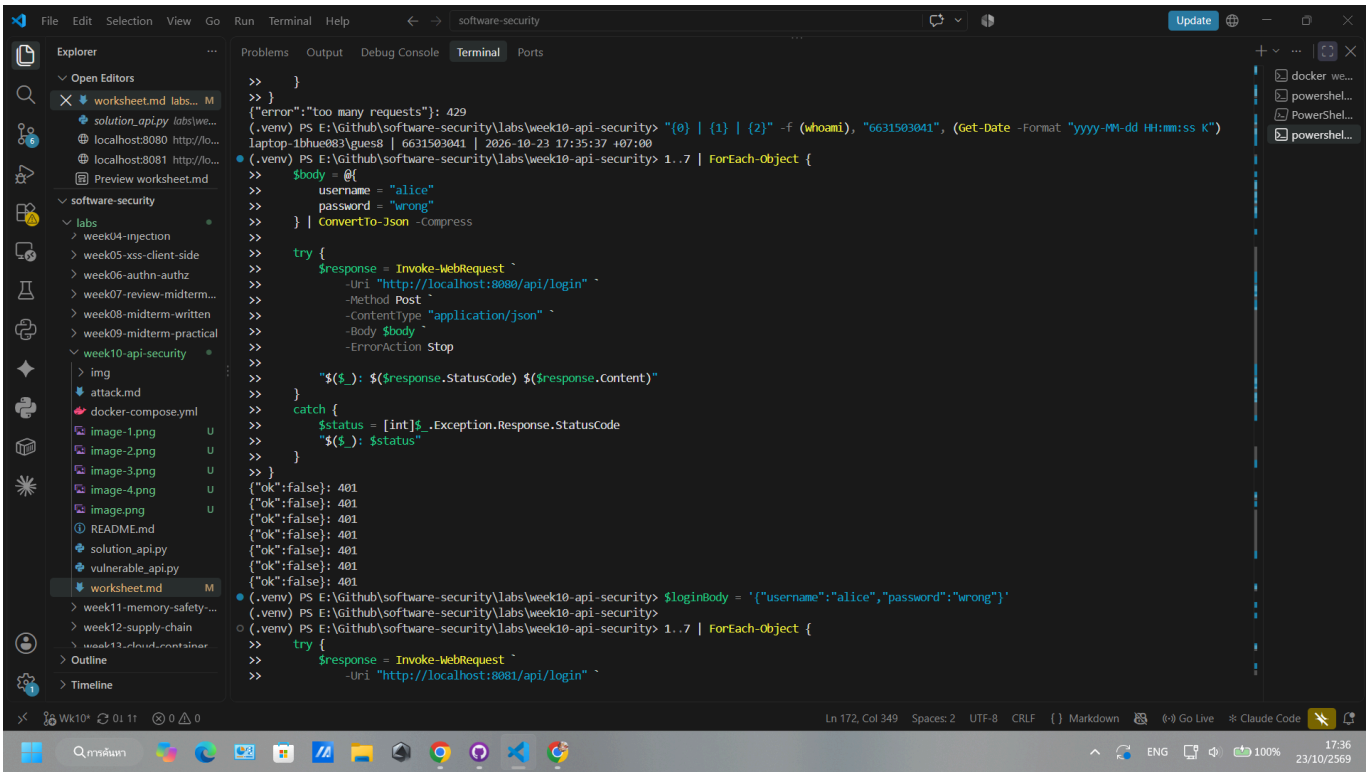


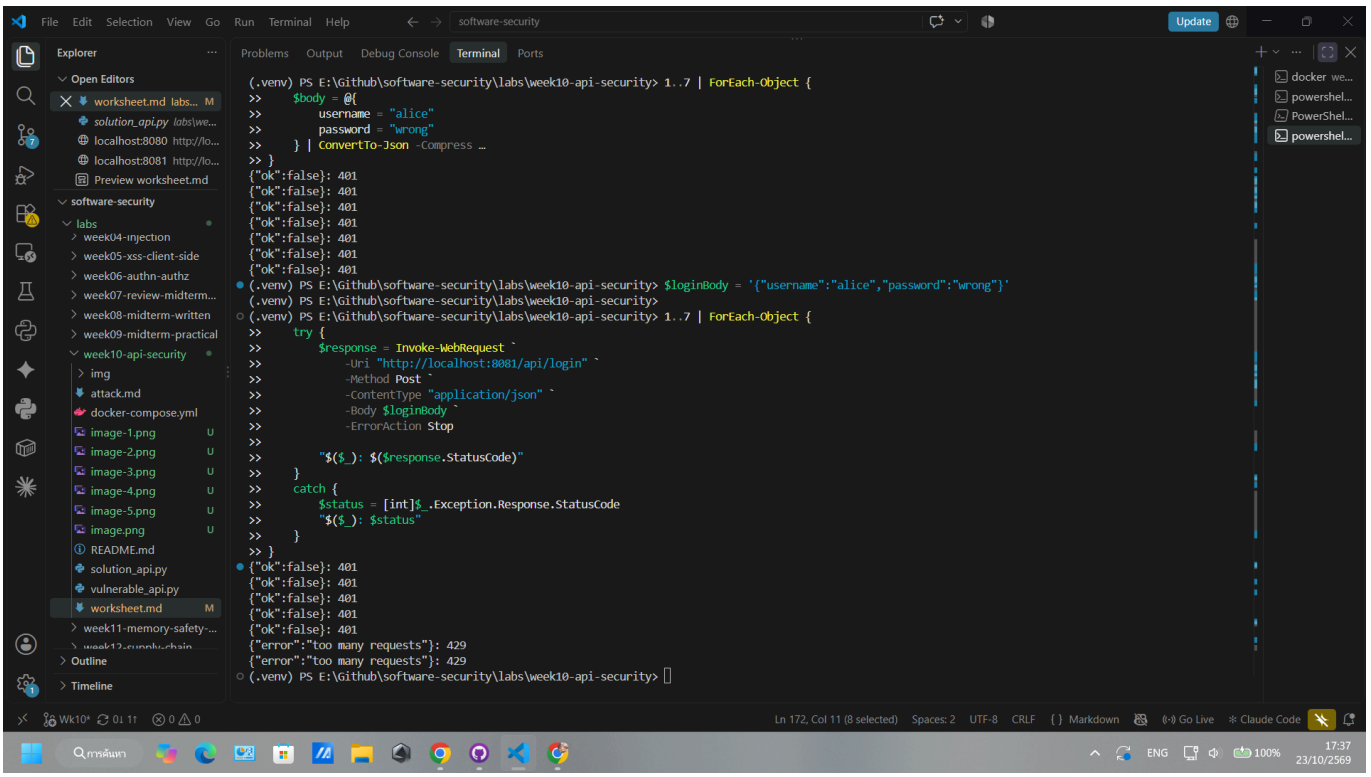

The API on port 8080 contains an API3:2023 vulnerability because it directly maps all fields from the client-submitted JSON body into the user object, allowing the client to arbitrarily set values for `is_admin` and `balance`. In contrast, the API on port 8081 employs an allow-list that accepts only the `username` and `password` fields, while the server itself assigns the `id`, `balance`, and `is_admin` values—resulting in `is_admin: false` and `balance: 0`.

## Task 3 — Unrestricted resource consumption: brute-force login (25 min)

**Goal:** Show `/api/login` has no throttle — API4:2023. **Steps:**

1. Run the brute-force loop from `attack.md` against `:8080` (guesses incl. `alice123`) — all attempts processed.
2. On `:8081` loop 7 times: `for i in $(seq 1 7); do curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8081/api/login -H "Content-Type: application/json" -d '{"username":"alice","password":"wrong"}'; done`
3. Confirm the 6th/7th attempt returns `429`. **Deliverable:** the `401 401 401 401 401 429 429` sequence + mitigation note.



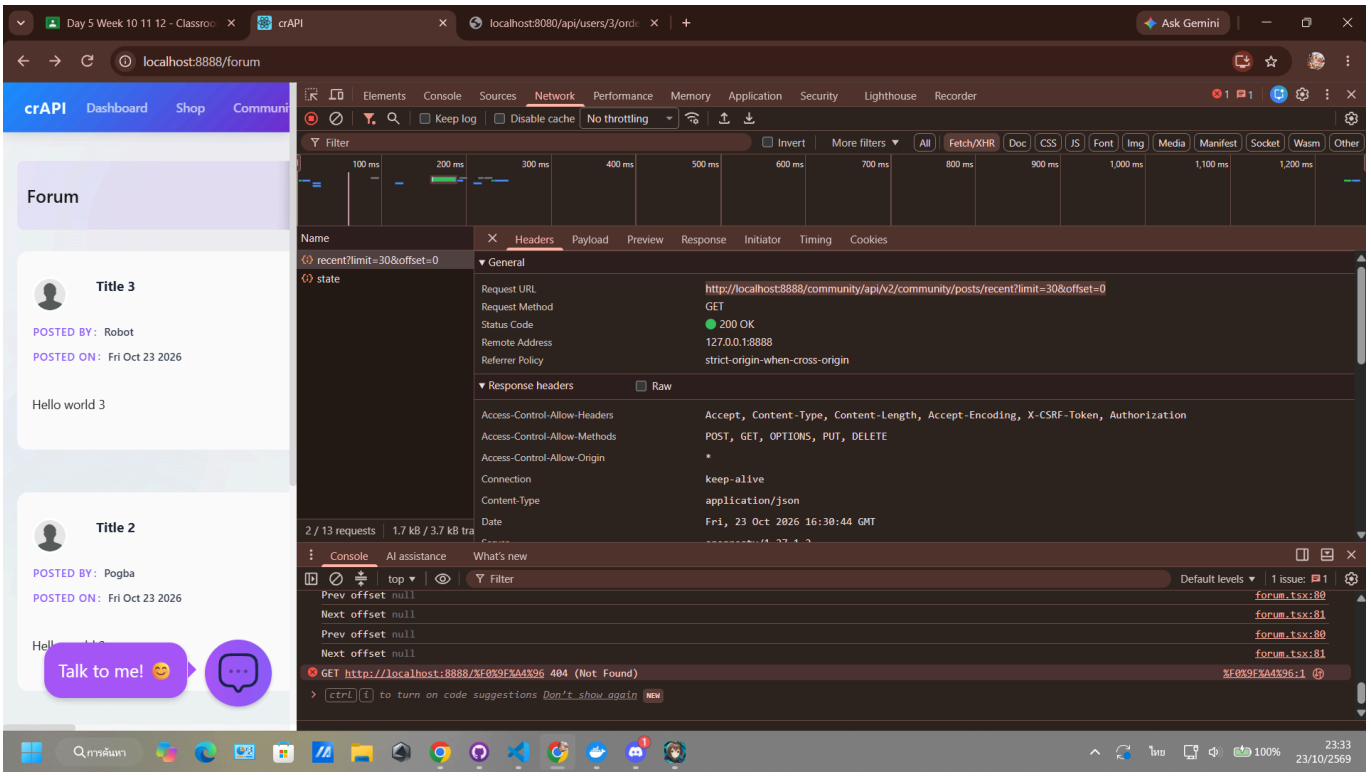

The insecure login endpoint is vulnerable to API4:2023 because it processes unlimited password guesses without throttling. This enables brute-force or credential-stuffing attacks and can consume server resources. The secure implementation applies a per-IP rate limit of five attempts per 60 seconds and returns HTTP 429 after the limit is exceeded.

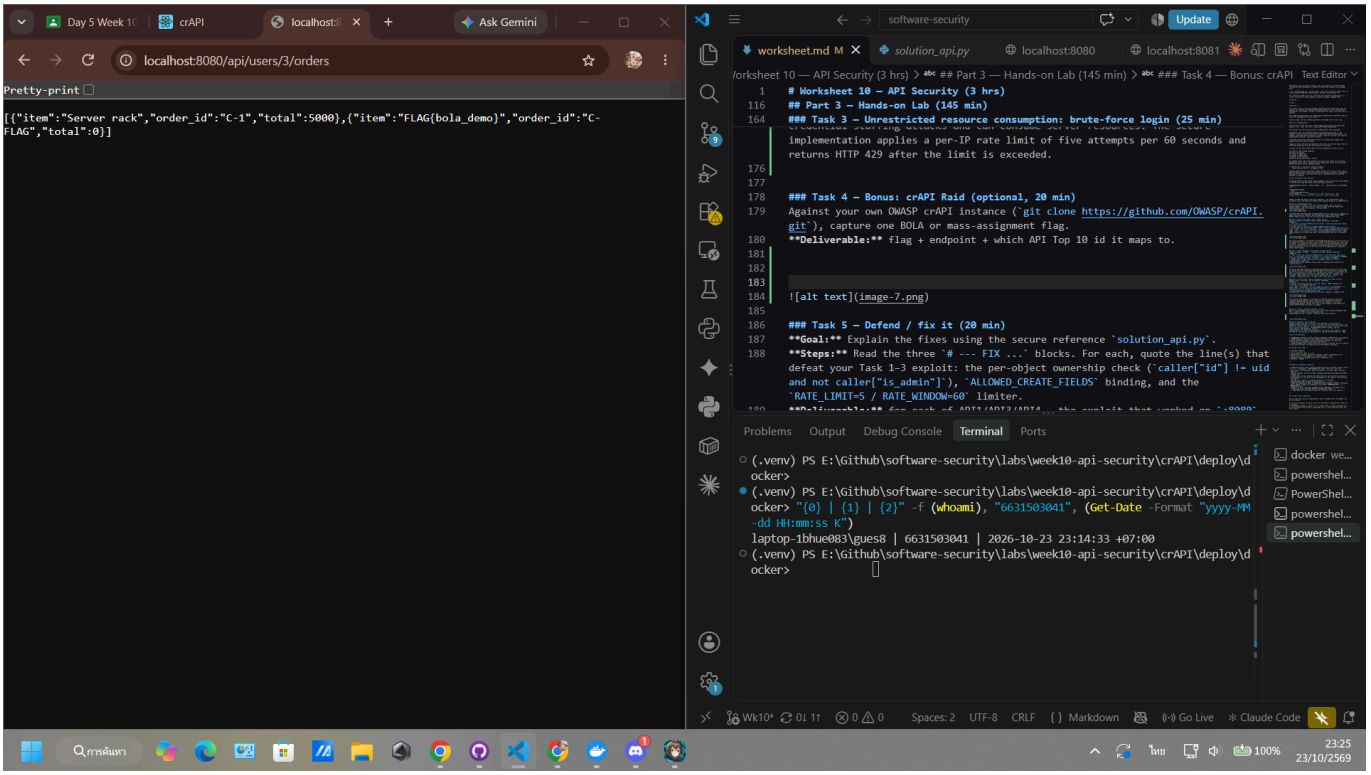
## Task 4 — Bonus: crAPI Raid (optional, 20 min)

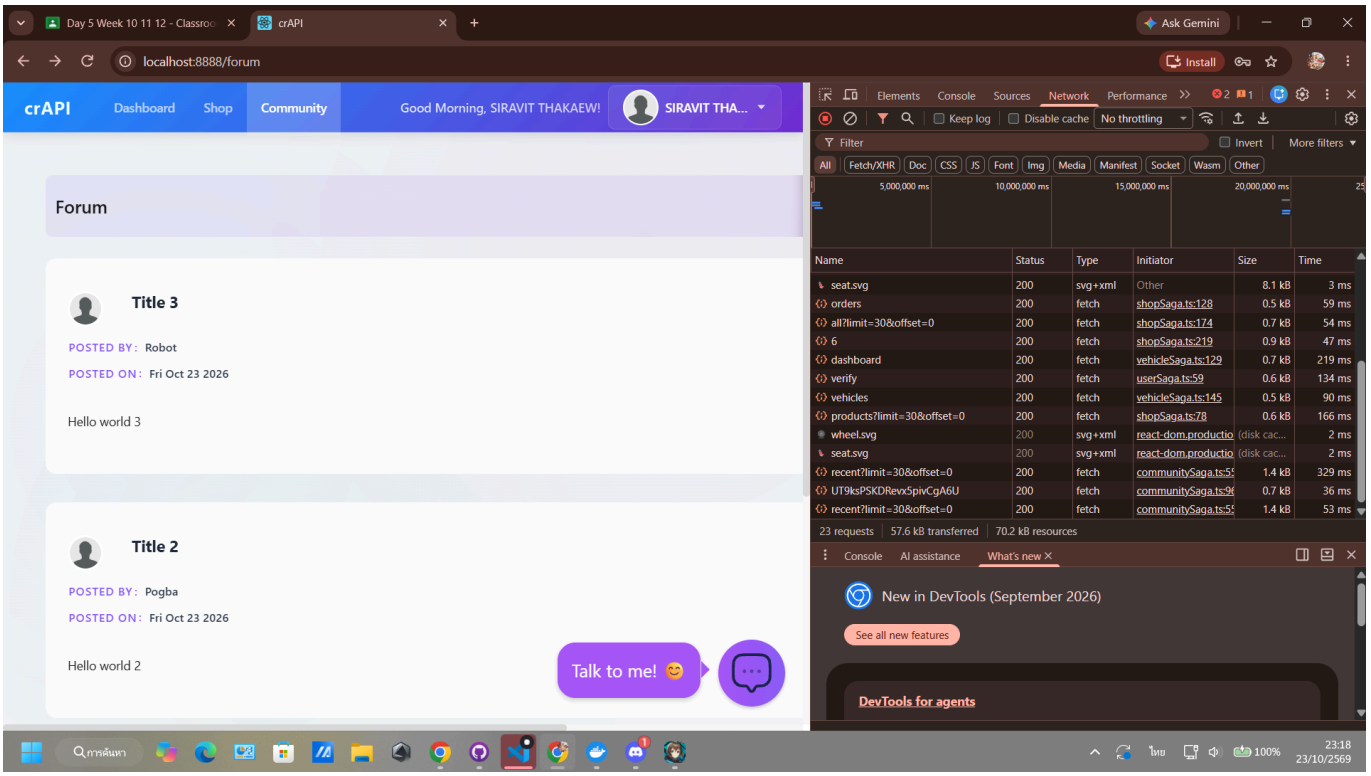
Against your own OWASP crAPI instance (`git clone https://github.com/OWASP/crAPI.git`), capture one BOLA or mass-assignment flag. **Deliverable:** flag + endpoint + which API Top 10 id it maps to.

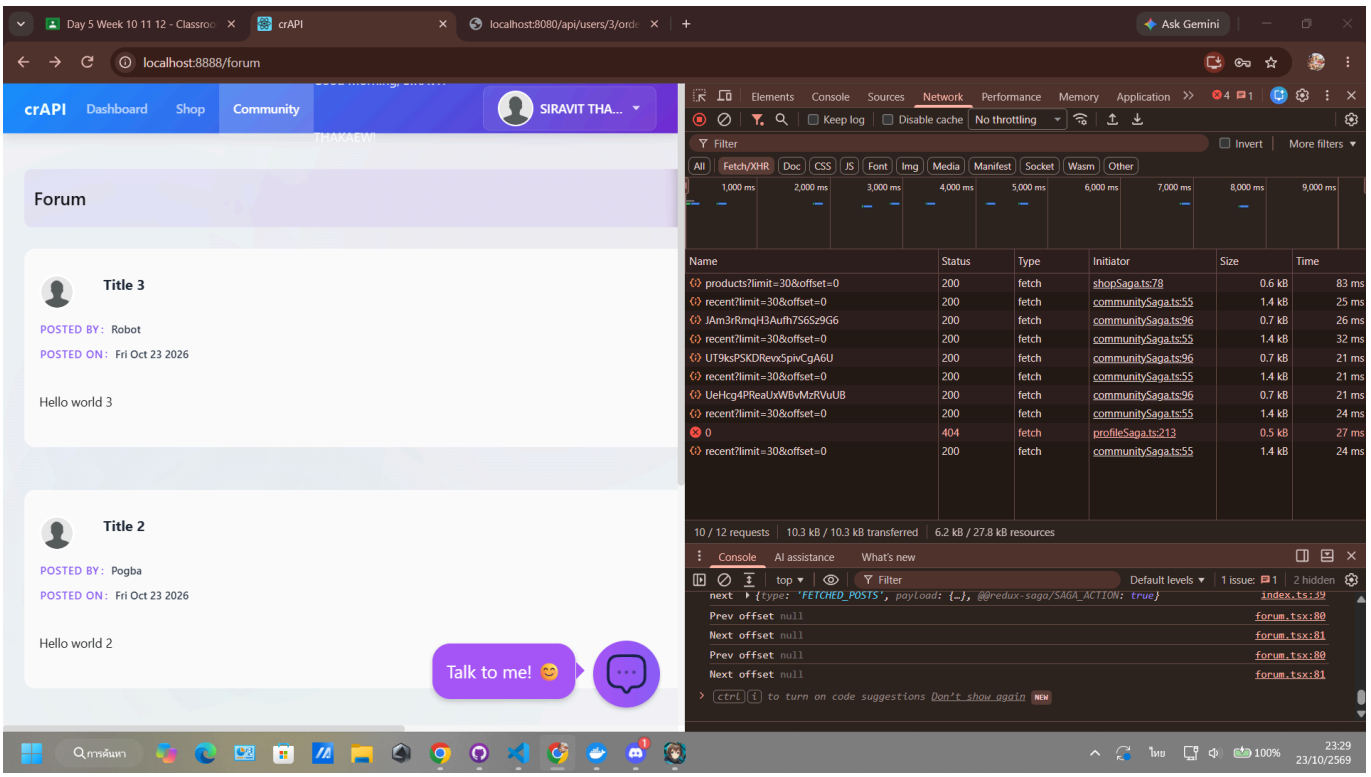
[{"item":"Server rack","order_id":"C-1","total":5000},{"item":"FLAG{bola_demo}","order_id":"C-FLAG","total":0}]

Endpoint: GET /community/api/v2/community/posts/recent?limit=30&offset=0 Full URL: http://localhost:8888/community/api/v2/community/posts/recent?limit=30&offset=0 Status: 200 OK






## Task 5 — Defend / fix it (20 min)

**Goal:** Explain the fixes using the secure reference `solution_api.py`. **Steps:** Read the three `# --- FIX ...` blocks. For each, quote the line(s) that defeat your Task 1–3 exploit: the per-object ownership check (`caller["id"] != uid and not caller["is_admin"]`), `ALLOWED_CREATE_FIELDS` binding, and the `RATE_LIMIT=5 / RATE_WINDOW=60` limiter. **Deliverable:** for each of API1/API3/API4 — the exploit that worked on `:8080`, the line in `solution_api.py` that blocks it, and the new HTTP status on `:8081`.

| OWASP ID | Exploit ที่ทำกับ :8080 | จุดแก้ไขใน solution_api.py | ผลบน :8081 |
|---|---|---|---|
| API1:2023 BOLA | ขอ /api/users/3/orders โดยไม่มีสิทธิ์ หรือให้ Alice ขอข้อมูลของ Carol | ตรวจ caller is None และ caller["id"] != uid and not caller["is_admin"] | ไม่มี identity ได้ 401; Alice ขอของ Carol ได้ 403; Alice ขอของตัวเองได้ 200 |
| API3:2023 Broken Object Property Level Authorization | ส่ง is_admin:true และ balance:1000000 ตอนสร้าง user | ใช้ ALLOWED_CREATE_FIELDS = {"username", "password"} และ server ตั้งค่า sensitive fields เอง | ได้ 201, แต่ is_admin:false, balance:0 |
| API4:2023 Unrestricted Resource Consumption | ยิง /api/login ซ้ำได้ไม่จำกัด | ใช้ RATE_LIMIT = 5, RATE_WINDOW = 60 และ rate_limited(...) | 5 ครั้งแรกได้ 401; ครั้งที่ 6 เป็นต้นไปได้ 429 |

1. API1 — BOLA จุดแก้ไขใน solution_api.py: caller = current_user()

if caller is None: return jsonify({"error": "authentication required"}), 401

if caller["id"] != uid and not caller["is_admin"]: return jsonify({"error": "forbidden"}), 403

2. API3 — Mass Assignment จุดแก้ไข: ALLOWED_CREATE_FIELDS = {"username", "password"} และ: user = {k: body[k] for k in ALLOWED_CREATE_FIELDS if k in body}

user.update({ "id": _next_id, "balance": 0, "is_admin": False })

3. API4 — Unrestricted Resource Consumption RATE_LIMIT = 5 RATE_WINDOW = 60 และ: if rate_limited(request.remote_addr or "unknown"): return jsonify({"error": "too many requests"}), 429

The secure reference applies three different controls for three different failure modes. API1 requires authentication and checks object ownership on every request; API3 uses an allow-list so clients can submit only safe fields; and API4 limits login attempts per IP. These controls stop the demonstrated exploits, but a production system should also use verified authentication tokens instead of trusting the lab-only X-User-Id header, distributed rate limiting, audit logging, and tests for authorization on every endpoint.

## Part 4 — Reflection

1. **Mapping:** complete a table — finding | endpoint | OWASP API id | fix applied.
2. **Real breach:** research an API authorization breach (e.g., the Optus or T-Mobile BOLA/IDOR incidents). Which API Top 10 id matches, and would the `solution_api.py` ownership check have prevented it?
3. **Best mitigation:** of object-level auth, allow-list binding, and rate limiting — which gives the most risk reduction for an API platform, and why?

## Grading rubric (100)

| Criterion | Weight |
|---|---|
| Lecture questions (Part 2) | 20 |

| Criterion | Weight |
|---|---|
| Exploitation + evidence (Tasks 1–4: commands, output, screenshots) | 40 |
| Defense (Task 5: fixes mapped to `solution_api.py`) | 25 |
| Reflection (Part 4: mapping, breach, best mitigation) | 15 |
| **Total** | **100** |

---

## Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: SUBMISSION.md).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*
  1. What did you do, and **why did the vulnerability work**?
  2. **Why does your fix actually stop it** — and what could still break it?

---

## 🤖 Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

---

## 🧠 Comprehension & Prompt (required)

**A. Explain in Plain English (EiPE).** In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

**B. Prompt Problem.** Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*